

Proje 3 & Proje 5

Rabia Şevval Yaşar
22290015

Proje 3

Github: <https://github.com/Rabia-23/BLM4522>

Video: https://drive.google.com/file/d/13wwYKh7KJKctdCbaVmHEwAsI-aKoJgGc/view?usp=drive_link

Proje 5

Github: <https://github.com/Rabia-23/BLM4522>

Video: <https://drive.google.com/file/d/1-Sv-X1e-5ck531xPRfFNwNRA7wdu91QI/view?usp=sharing>

PROJE 3 -VERITABANI GUVENLIGI VE ERISIM KONTROLU

1. GIRIS

İnternet üzerinden Northwind veritabanı indirilmiştir ve bu veritabanının güvenlik durumu sorgular yoluyla incelenmiştir.

Bu inceleme sonucunda üç adet önemli güvenlik açığı belirlenmiştir. Bu açıklar - erişim kontrolünün bulunmaması, çalışanlara ait özel verilerin şifrelenmemiş olması ve kullanıcı hareketlerinin kaydedilmemesidir. Bu eksiklikler giderilmiştir ve beş farklı seviyeden oluşan bir güvenlik sistemi oluşturulmuştur.

Güvenlik açıkları kasıtlı olarak eklenmemiştir. Veritabanının ilk halinde var olan teknik eksiklikler saptanmıştır ve bunlar düzeltilmiştir.

Araçlar: PostgreSQL 14, Northwind Database, pgcrypto, psql

1.1. UYGULANAN BES GUVENLIK KATMANI

Katman 1

Rol Tabanlı Erişim Kontrolü (RBAC): Kullanıcı ve yetki yönetimi

Katman 2

Veri Sifreleme: pgcrypto ile AES-128 sifreleme

Katman 3

SQL Injection Korunması: Parametrelili sorgular

Katman 4

Audit Log: Trigger tabanlı aktivite izleme

Katman 5

Row Level Security (RLS): Satir düzeyinde erişim kontrolü

2. VERİTABANI TANITIMI VE KURULUM

Northwind, bir ticaret şirketinin müşteri, sipariş ve ürün verilerini içeren açık kaynaklı bir örnek veritabanıdır. 14 tablo içermektedir.

2.1. KURULUM

```
psql postgres -c "CREATE DATABASE northwind;"
psql -d northwind -f northwind.sql
```

3. MEVCUT GÜVENLİK AÇIKLARI TESPİTİ

Veritabanı kurulduktan sonra ilk adım olarak mevcut güvenlik durumu dört sorguyla sistematik şekilde belgelenmiştir.

3.1. ERİŞİM KONTROLÜ DURUMU

```
SELECT * FROM pg_roles;
```

Bulgu: Yalnızca superuser hesabı mevcut. Hiç rol tanımlanmamış.

```
SELECT grantee, privilege_type, table_name
FROM information_schema.role_table_grants
WHERE table_name IN ('employees', 'customers', 'orders')
AND grantee NOT IN ('rabiasevvalyasar', 'PUBLIC');
```

Bulgu: 0 satır döndü — hiçbir tabloda yetki kısıtlaması tanımlanmamış.

3.2. ŞİFRELEME DURUMU

```
SELECT table_name, column_name
FROM information_schema.columns
WHERE column_name ILIKE '%password%'
OR column_name ILIKE '%encrypt%';
```

Bulgu: Northwind tablolarında hiçbir şifreli alan yok. employees tablosundaki hassas kişisel veriler düz metin.

3.3. RLS VE AKTIVITE IZLEME DURUMU

```
SELECT * FROM pg_policies;
```

Bulgu: 0 satır — Row Level Security hiç yapılandırılmamış, aktivite izleme yok.

4. KATMAN 1: ROL TABANLI ERISIM KONTROLU

'En az yetki' prensibi gereği her kullanıcıya sadece ihtiyacı kadar yetki verilmiştir.

```
CREATE ROLE readonly_role;
CREATE ROLE dataentry_role;
CREATE ROLE admin_role;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO dataentry_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;

CREATE USER analyst WITH PASSWORD 'analyst123';
CREATE USER dataentry WITH PASSWORD 'dataentry123';
CREATE USER dbadmin WITH PASSWORD 'dbadmin123';

GRANT readonly_role TO analyst;
GRANT dataentry_role TO dataentry;
GRANT admin_role TO dbadmin;
```

4.1. ERISIM TESTI

```
SET ROLE analyst;
SELECT * FROM customers LIMIT 3;    -- Basarili
INSERT INTO customers (customer_id, company_name)
VALUES ('TEST', 'Test');            -- HATA: permission denied
RESET ROLE;
```

Sonuç: Analyst sadece okuma yapabiliyor — erişim kontrolü çalışıyor.

Kullanıcı	Rol	İzinler
analyst	readonly_role	Sadece SELECT
dataentry	dataentry_role	SELECT, INSERT, UPDATE
dbadmin	admin_role	Tum yetkiler

5. KATMAN 2: HASSAS VERİ SIFRELEME

employees tablosundaki home_phone alanı düz metin olarak saklanıyordu. pgcrypto eklentisiyle AES-128 şifreleme uygulanmıştır.

```
CREATE EXTENSION pgcrypto;

ALTER TABLE employees ADD COLUMN home_phone_encrypted TEXT;

UPDATE employees
SET home_phone_encrypted = encode(
    encrypt(home_phone::bytea, 'gizlianahtar'::bytea, 'aes'),
    'base64'
)
WHERE home_phone IS NOT NULL;

-- Duz metin sutunu gizle
UPDATE employees SET home_phone = '***GIZLI***';
```

5.1. SIFRE COZME DOGRULAMASI

```
SELECT first_name,
       convert_from(
           decrypt(decode(home_phone_encrypted, 'base64'),
                  'gizlianahtar'::bytea, 'aes'), 'UTF8'
       ) AS gercek_telefon
FROM employees LIMIT 3;
```

Sonuç: Doğru anahtar ile şifre başarıyla çözüldü. Yanlış anahtar kullanıldığında veri okunamaz hale gelir.

6. KATMAN 3: SQL INJECTION KORUNMASI

SQL Injection, OWASP Top 10 listesinde yer alan en yaygın veritabanı saldırılarından biridir.

6.1. SAVUNMASIZ SORGU

```
SELECT * FROM customers
WHERE customer_id = '' OR '1'='1'
LIMIT 5;
```

Sonuç: Tüm müşteri tablosu döndü – ciddi güvenlik açığı

6.2. PARAMETRELİ SORGU İLE KORUNMA

```
PREPARE guvenli_sorgu AS
SELECT * FROM customers WHERE customer_id = $1;

EXECUTE guvenli_sorgu('ALFKI');           -- Normal kullanım
EXECUTE guvenli_sorgu('' OR ''1'=''1'); -- Saldiri denemesi
```

Sonuç: Saldırı denemesi 0 satır döndürdü – SQL Injection engellendi.

7. KATMAN 4: AUDIT LOG SİSTEMİ

Veritabanında yapılan hiçbir işlem izlenmiyordu. Trigger tabanlı audit log sistemi kurularak her INSERT, UPDATE ve DELETE işlemi kayıt altına alınmıştır.

```
CREATE TABLE audit_log (
    id SERIAL PRIMARY KEY,
    tablo_adi TEXT, islem TEXT,
    kullanıcı TEXT, eski_deger TEXT,
    yeni_deger TEXT,
    islem_zamani TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION audit_trigger_func()
RETURNS TRIGGER AS $$
```

```

BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (tablo_adi, islem, kullanıcı, yeni_deger)
        VALUES (TG_TABLE_NAME, 'INSERT', current_user, row_to_json(NEW)::text);
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (tablo_adi, islem, kullanıcı, eski_deger, yeni_deger)
        VALUES (TG_TABLE_NAME, 'UPDATE', current_user,
            row_to_json(OLD)::text, row_to_json(NEW)::text);
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (tablo_adi, islem, kullanıcı, eski_deger)
        VALUES (TG_TABLE_NAME, 'DELETE', current_user, row_to_json(OLD)::text);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER customers_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();

```

7. KATMAN 4: AUDIT LOG SİSTEMİ

SQL Injection, OWASP Top 10 listesinde yer alan en yaygın veritabanı saldırılarından biridir.

7.1. TEST VE DOGRULAMA

```

INSERT INTO customers (customer_id, company_name, contact_name, country)
VALUES ('TEST1', 'Test Sirketi', 'Ali Yilmaz', 'Turkey');
UPDATE customers SET contact_name = 'Ayse Yilmaz' WHERE customer_id = 'TEST1';
DELETE FROM customers WHERE customer_id = 'TEST1';

SELECT id, tablo_adi, islem, kullanıcı, islem_zamani FROM audit_log;

```

8. KATMAN 5: ROW LEVEL SECURITY (RLS)

Row Level Security, aynı tabloya erişimi olan kullanıcıların sadece belirli satırları görmesini sağlar. Bu özellik veritabanında farklı kullacıların sadece kendi verilerini görmesi gereken senaryolarda kritik önem taşır.

8.1. SENARYO

Northwind'in orders tablosunda 21 farklı ülkeye ait sipariş bulunmaktadır.

Senaryo: Her bölge sorumlusu sadece kendi ülkesinin siparişlerini görebilsin.

```
-- Bölge sorumlusu kullanicilari
CREATE USER us_manager WITH PASSWORD 'us123';
CREATE USER de_manager WITH PASSWORD 'de123';
CREATE USER br_manager WITH PASSWORD 'br123';

GRANT CONNECT ON DATABASE northwind TO us_manager, de_manager, br_manager;
GRANT USAGE ON SCHEMA public TO us_manager, de_manager, br_manager;
GRANT SELECT ON orders TO us_manager, de_manager, br_manager;
```

8.2. RLS POLİTİKALARININ TANIMLANMASI

```
-- RLS'i etkinleştir
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Her kullanıcı için politika
CREATE POLICY us_only ON orders
  FOR SELECT TO us_manager
  USING (ship_country = 'USA');

CREATE POLICY de_only ON orders
  FOR SELECT TO de_manager
  USING (ship_country = 'Germany');

CREATE POLICY br_only ON orders
  FOR SELECT TO br_manager
  USING (ship_country = 'Brazil');

-- Politikaları listele
SELECT tablename, policyname, roles, cmd, qual
FROM pg_policies WHERE tablename = 'orders';
```

8.3. RLS TEST SONUCLARI

```
-- Super kullanıcı: tüm ülkeler görünüyor
SELECT ship_country, COUNT(*) FROM orders
GROUP BY ship_country ORDER BY COUNT(*) DESC LIMIT 5;

SET ROLE us_manager;
SELECT ship_country, COUNT(*) FROM orders GROUP BY ship_country;
-- Sonuç: Sadece USA (122 sipariş)
RESET ROLE;

SET ROLE de_manager;
SELECT ship_country, COUNT(*) FROM orders GROUP BY ship_country;
-- Sonuç: Sadece Germany (122 sipariş)
RESET ROLE;

SET ROLE br_manager;
SELECT ship_country, COUNT(*) FROM orders GROUP BY ship_country;
-- Sonuç: Sadece Brazil (83 sipariş)
RESET ROLE;
```

Sonuç: RLS politikaları başarıyla çalışıyor. Her kullanıcı sadece yetkili olduğu ülkenin siparişlerini görebiliyor.

9. SONUC VE DEGERLENDIRME

9.1. TESPIT EDILEN ACIKLAR VE UYGULANAN COZUMLER

Tespit	Cozum	Sonuc
Erisim kontrolu yok	RBAC — 3 rol, 3 kullanıcı	Her kullanıcı sadece gerekli yetkiyle çalışır
Hassas veri şifrelenmemiş	pgcrypto AES-128	home_phone şifrelendi
Aktivite izleme yok	Trigger tabanlı audit log	Her işlem kayıt altında
SQL Injection acığı	Parametrelili sorgular	Saldırı 0 satır dondurdu
RLS politikası yok	Satır düzeyinde erişim kontrolü	Her yönetici sadece kendi ülkesini görüyor

PROJE 5 - VERİ TEMİZLEME VE ETL SURECLERİ TASARIMI

1. GİRİŞ

Proje kapsamında günlük yaşamda verilerde görülen niteliksel eksiklikleri ortadan kaldırmak için ETL (Extract, Transform, Load) işlemleri yürütülmüştür.

Projede, internet ortamında bulunan ve Chinook adını taşıyan mevcut bir örnek veritabanı sisteme dahil edilmiştir. Veritabanının yapısında halihazırda bulunan veri kalitesi problemleri bu süreçte tespit edilmiştir. Bu verilere dışarıdan bilinçli bir şekilde hata dahil edilmemiştir. Veritabanının kendi özgün yapısında var olan bu sorunlar, düzenli olarak çalıştırılan sorgular yardımıyla incelenmiş ve veriler ayıklanmıştır.

Araçlar: PostgreSQL 14, Chinook Database, psql

1.1. ETL NEDİR?

Extract (Çıkar)

Kaynak verideki kalite sorunlarını keşfetmek ve raporlamak.

Transform (Dönüştür)

Tespit edilen sorunları temizlemek, standartlaştırmak ve eksik alanları doldurmak.

Load (Yükle)

Temizlenmiş veriyi hedef tabloya aktarmak.

2. VERİTABANI TANITIMI VE KURULUM

Chinook, bir müzik satış şirketini simüle eden açık kaynaklı bir örnek veritabanıdır. 11 tablo içermekte olup ETL sürecinde customer tablosu kullanılmıştır.

2.1. KURULUM

```
psql postgres -c "CREATE DATABASE chinook;"  
psql -d chinook -f Chinook_PostgreSql.sql
```

2.2. CUSTOMER TABLOSU

Customer tablosu 59 müşteri kaydını ve 13 sütunu içermektedir.

3. EXTRACT ASAMASI – TUM SUTUNLARIN SISTEMATIK TARANMASI

Extract aşamasında customer tablosunun 12 sütununun tamamı tek bir sorguyla taranmıştır. Bu yaklaşım hiçbir sorunun gözden kaçmasını engeller ve kapsamlı bir veri kalitesi analizi sağlar.

3.1. KAPSAMLI TARAMA SORGUSU

```
SELECT  
  COUNT(*) AS toplam,  
  COUNT(*) FILTER (WHERE first_name IS NULL  
                    OR first_name = '') AS bos_isim,  
  COUNT(*) FILTER (WHERE last_name IS NULL  
                    OR last_name = '') AS bos_soyisim,  
  COUNT(*) FILTER (WHERE email IS NULL) AS null_email,  
  COUNT(*) FILTER (WHERE phone IS NULL) AS null_phone,  
  COUNT(*) FILTER (WHERE fax IS NULL) AS null_fax,  
  COUNT(*) FILTER (WHERE company IS NULL) AS null_company,  
  COUNT(*) FILTER (WHERE address IS NULL  
                    OR address = '') AS bos_adres,  
  COUNT(*) FILTER (WHERE city IS NULL  
                    OR city = '') AS bos_sehir,  
  COUNT(*) FILTER (WHERE country IS NULL  
                    OR country = '') AS bos_ulke,  
  COUNT(*) FILTER (WHERE state IS NULL  
                    OR state = '') AS bos_state,  
  COUNT(*) FILTER (WHERE postal_code IS NULL  
                    OR postal_code = '') AS bos_posta_kodu  
FROM customer;
```

3.2. TARAMA SONUCLARI

Sutun	Sorunlu Kayit	Durum
first_name	0	Temiz
last_name	0	Temiz
email	0	Temiz
address	0	Temiz
city	0	Temiz
country	0	Temiz
phone	1	Sorun tespit edildi — duzeltilecek
company	49	Sorun tespit edildi — siniflandirilacak
state	29	Cografî gerceklik — standartlastirilacak
postal_code	4	Sorun tespit edildi — isaretlenecek
fax	47	Opsiyonel alan — mudahale edilmeyecek

Sonuc: 12 sutundan 6'si tamamen temiz, 5'inde sorun tespit edilmistir

3.3. DETAYLI ANALIZ SORGULARI

```
SELECT customer_id, first_name, last_name, country, phone
FROM customer WHERE phone IS NULL;
```

Bulgu: 1 müşteri telefon bilgisi olmadan kayıtlı.

```
SELECT country, COUNT(*) as sayi
FROM customer
WHERE state IS NULL OR state = ''
GROUP BY country ORDER BY sayi DESC;
```

Bulgu: Fransa (5), Almanya (4), İngiltere (3) gibi Avrupa ülkelerinde eyalet sistemi bulunmamaktadır. Bu değer eksikliği bir hata değil, coğrafi bir gerçekliktir.

```
SELECT customer_id, first_name, last_name, country, postal_code
FROM customer
WHERE postal_code IS NULL OR postal_code = '';
```

Bulgu: 4 müşterinin posta kodu eksik — gerçek bir veri kalitesi sorunu.

```
SELECT
  COUNT(*) FILTER (WHERE phone LIKE '+%')      AS uluslararası_format,
  COUNT(*) FILTER (WHERE phone NOT LIKE '+%'
                    AND phone IS NOT NULL)      AS yerel_format,
  COUNT(*) FILTER (WHERE phone IS NULL)        AS bos
FROM customer;
```

Bulgu: Tüm telefonlar '+' ile başlıyor — format tutarlı ve uluslararası standarda uygun. Bu olumlu bir bulgudur.

```
SELECT country, COUNT(*) as toplam,
  COUNT(fax) as fax_olan,
  COUNT(*) - COUNT(fax) as fax_olmayan
FROM customer
GROUP BY country HAVING COUNT(*) - COUNT(fax) > 0
ORDER BY fax_olmayan DESC;
```

Bulgu: 47 müşterinin fax numarası yok. Fax opsiyonel bir iletişim kanalı olduğu için bu durum hata sayılmaz — müdahale edilmeyecektir.

4. TRANSFORM ASAMASI – TEMİZLEME VE STANDARTLASTIRMA

Transform aşamasında her sorun için farklı bir strateji belirlenmiştir. Temel ilke: her NULL değer bir hata değildir. Bazı alanlar doğası gereği boş olabilir.

4.1. NULL TELEFONU DÜZELT

Tek müşterinin telefonu NULL. Silmek yerine 'Bilgi Yok' ile işaretliyoruz — müşterinin diğer bilgileri geçerlidir.

```
UPDATE customer
SET phone = 'Bilgi Yok'
WHERE phone IS NULL;
-- Sonuc: UPDATE 1
```

4.2. EKSİK SİRKET BİLGİSİNİ SINIFLANDIR

49 müşterinin şirket bilgisi eksik. Bu müşteriler bireysel müşteri olabilir – 'Bireysel Musteri' değeri ile sınıflandırıyoruz.

```
UPDATE customer
SET company = 'Bireysel Musteri'
WHERE company IS NULL;
-- Sonuc: UPDATE 49
```

4.3. AVRUPA'DA STATE ALANINI STANDARTLASTIR

Avrupa ülkelerinde eyalet sistemi bulunmadığı için state alanı doğal olarak boş. Tutarlılık için 'N/A' değeri atıyoruz.

```
UPDATE customer
SET state = 'N/A'
WHERE (state IS NULL OR state = '')
AND country NOT IN ('USA', 'Canada', 'Brazil', 'Australia');
-- Sonuc: UPDATE 29
```

4.4. EKSİK POSTA KODLARINI İSARETLE

Portekiz, İrlanda ve Şili'den 4 müşterinin posta kodu eksik. Gerçek bir veri kalitesi sorunu olduğu için 'Bilinmiyor' ile işaretliyoruz.

```
UPDATE customer
SET postal_code = 'Bilinmiyor'
WHERE postal_code IS NULL OR postal_code = '';
-- Sonuc: UPDATE 4
```

4.5. FAX ALANI – MUDAHALE EDİLMEDİ

Fax bilgisi opsiyonel bir iletişim kanalıdır. NULL olması veri hatası değildir. Bu alan olduğu gibi bırakılmış ve karar aşağıda belgelenmiştir.

```
UPDATE customer
SET state = 'N/A'
WHERE (state IS NULL OR state = '')
AND country NOT IN ('USA', 'Canada', 'Brazil', 'Australia');
-- Sonuc: UPDATE 29
```

4.6. TRANSFORM OZETI

Alan	Uygulanan Strateji	Etkilenen Kayıt
phone	'Bilgi Yok' ile işaretle	1
company	'Bireysel Müşteri' olarak sınıflandır	49
state	Avrupa için 'N/A' ile standartlaştır	29
postal_code	'Bilinmiyor' ile işaretle	4
fax	Müdahale edilmedi (opsiyonel alan)	—

5. LOAD ASAMASI – TEMİZ TABLOYU OLUSTURMA

Load asamasında temizlenmiş veri yeni bir tabloya aktarılmıştır. Ham veri korunmaktadır – profesyonel ETL sistemlerinde bu ayırım kritik önem taşır.

5.1. CUSTOMER_CLEAN TABLOSU

```
CREATE TABLE customer_clean AS
SELECT * FROM customer
WHERE phone != 'Bilgi Yok'
AND postal_code != 'Bilinmiyor';
```

5.2. VERİ KALİTESİ RAPORU

```
SELECT 'Ham veri (toplam)' AS aciklama, COUNT(*) AS sayi FROM customer
UNION ALL
SELECT 'Temiz veri', COUNT(*) FROM customer_clean
UNION ALL
SELECT 'Telefon duzeltilen', COUNT(*) FROM customer
WHERE phone = 'Bilgi Yok'
UNION ALL
SELECT 'Sirket eklenen', COUNT(*) FROM customer
WHERE company = 'Bireysel Musteri'
UNION ALL
SELECT 'State standartlastirilan', COUNT(*) FROM customer
WHERE state = 'N/A'
UNION ALL
SELECT 'Posta kodu isaretlenen', COUNT(*) FROM customer
WHERE postal_code = 'Bilinmiyor'
UNION ALL
SELECT 'Fax yok (opsiyonel)', COUNT(*) FROM customer
WHERE fax IS NULL;
```

Metrik	Deger
Ham veri (toplam)	59 satir
Temiz veri	54 satir
Telefon duzeltilen	1 kayit
Sirket bilgisi eklenen	49 kayit
State standartlastirilan	29 kayit (N/A)
Posta kodu isaretlenen	4 kayit
Fax yok (opsiyonel, birakildi)	47 kayit

5.3. TEMİZ TABLO DOGRULAMASI

```
SELECT
COUNT(*) AS toplam,
COUNT(phone) AS telefon_dolu,
COUNT(company) AS sirket_dolu,
COUNT(state) AS state_dolu,
COUNT(postal_code) AS posta_dolu
FROM customer_clean;
```

6. SONUC VE DEGERLENDIRME

6.1. EXTRACT BULGULARININ OZETI

12 sütunun tamamının taranması sonucunda:

- 6 sütun tamamen temiz çıkmıştır: first_name, last_name, email, address, city, country.
- 5 sütunda sorun tespit edilmiş, her biri için farklı bir strateji uygulanmıştır.
- Telefon formatları uluslararası standarda tamamen uygundur – bu olumlu bir veri kalitesi göstergesidir.

6.2. VERI KALITESI MUHAKEMESI

Bu projede en önemli öğrenilen ders şudur: Her NULL değer bir hata değildir. ETL sürecinde bu ayrımı yapabilmek kritik öneme sahiptir

Durum	Karar	Gerekçesi
Fax NULL	Müdahale edilmedi	Opsiyonel iletişim kanalı
State NULL (Avrupa)	N/A ile standartlaştırıldı	Eyalet sistemi yok
Phone NULL	'Bilgi Yok' ile işaretlendi	Eksik zorunlu bilgi
postal_code bos	'Bilinmiyor' ile işaretlendi	Eksik adres bilgisi

6.3. KULLANILAN SQL KOMUTLARI

Komut	Kullanım Amacı
SELECT / COUNT / FILTER	Tüm sütunları sistematik tarama
GROUP BY / HAVING	Kategori bazlı detaylı analiz
UPDATE	Veri düzeltme ve standartlaştırma
CREATE TABLE AS SELECT	Temiz tabloyu oluşturma (Load adımı)
UNION ALL	Önce/sonra karşılaştırma raporu